SYSTEM INTEGRATION

Exam Project, EASV PB Software 2025

Synopsis

Architecting for Scalability and Resilience: A Case Study in Decomposing an E-commerce Monolith into the SynopsisSI Microservice Platform using API Gateways, Event-Driven Messaging, and Polyglot Persistence.

Klaus Hviid

28-05-2025

SYSTEM INTEGRATION

Indhold

The GitHub repository is available here: https://github.com/zosodk/SynopsisSI

It is created as a collaborative project for two courses: Databases for Developers and System Integration.

This repository will also serve as a tool for the practical examination of the Distributed Large Systems course, which requires practical examples of the technologies covered in the course.

1. Introduction

Traditional monolithic architectures, where all components are developed and deployed as a single unit, have long been standard in software development. In e-commerce platforms, this means functionalities like product listings, order processing, user management, and payment handling are tightly coupled within one large codebase. While initially simple, this approach faces challenges as the platform scales, including difficulties in scaling specific features independently, reduced development agility, slower deployment cycles, inflexibility in adopting new technologies, and poor fault isolation.

To address these limitations, this paper explores transitioning to a microservice architecture through the case study of SynopsisSI, a backend system for a modern e-commerce platform for second-hand item transactions. The project aims to demonstrate how decomposing a monolithic application into smaller, independent microservices can lead to a more scalable, resilient, and maintainable system, allowing tailored development, deployment, and scaling strategies for different business capabilities.

The central problem addressed is the effective integration of these distributed microservices. While decomposition offers benefits, it introduces complexities in service discovery, reliable communication, and external client access management. The core thesis question is:

How can an API Gateway (YARP) and event-driven messaging (RabbitMQ with MassTransit) be designed and implemented to integrate core e-commerce microservices (ListingService, OrderService, UserService within the SynopsisSI platform), ensuring controlled external access, promoting loose coupling, and enabling resilient inter-service communication?

2. Architectural Overview

The SynopsisSI platform is conceived as a distributed system, moving away from traditional monolithic constraints to embrace a microservice architecture. This architectural choice is driven by the need for enhanced scalability to handle fluctuating user loads, improved agility for faster development and deployment cycles of individual features, and increased resilience by isolating faults within specific service boundaries. Furthermore, a microservice approach allows for technology diversity, enabling each service to potentially leverage the most appropriate tools and data stores for its unique requirements, although the current implementation prioritizes a consistent .NET 9 stack for core services.

Rationale for Choosing a Microservice Architecture

The primary motivation for adopting microservices for SynopsisSI is to build a system capable of independent evolution and scaling. E-commerce platforms often experience variable loads across different functionalities; for example, product Browse (ListingService) might have significantly different traffic patterns and scaling needs than order processing (OrderService) or user management (UserService). A microservice architecture allows each of these components to be scaled independently, optimizing resource utilization. It also facilitates smaller, focused

development teams, reduces the complexity of individual codebases, and allows for independent deployment pipelines, thereby accelerating the delivery of new features and updates while minimizing the risk associated with large, monolithic releases. The inherent fault isolation also means that an issue in a non-critical service is less likely to impact the core functionality of the entire platform.

Service Identification and Responsibilities

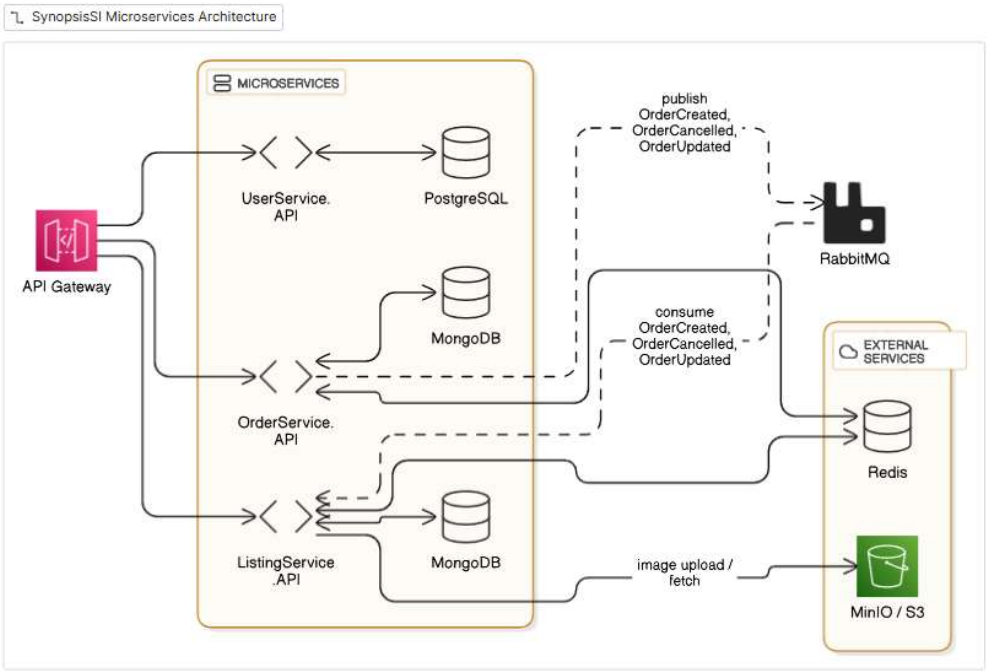
For the initial development of SynopsisSI, the system has been decomposed into several core microservices, each with a clearly defined set of responsibilities aligning with distinct business capabilities:

ListingService: This service is the definitive authority for all product listing information. Its responsibilities include managing the creation, retrieval, updating, and deletion (or delisting) of items. It handles all data pertinent to listings, such as titles, descriptions, pricing, categorization, condition details, image references (as URLs), and geographical location. Given the potentially diverse and unstructured nature of item-specific attributes for second-hand goods, this service is designed with data flexibility in mind.

OrderService: This service orchestrates the entire lifecycle of customer orders. Its duties encompass the creation of new orders from cart or listing data, processing the items within an order, calculating total amounts, and managing the progression of order statuses (e.g., from pending payment to paid, shipped, and delivered). A key interaction for this service is publishing events upon successful order placement to notify other interested services.

UserService (incorporating Authentication Logic for this iteration): This service is central to managing user identity and profile information. It handles user registration, storage and retrieval of user profiles (including contact details and addresses), and, for the current scope, manages user authentication by validating credentials and generating JSON Web Tokens (JWTs) for secure access to the platform's APIs.

High-Level System Diagram



The API Gateway

In the SynopsisSI microservice architecture, the API Gateway serves as the crucial front door, abstracting the complexities of the distributed backend from client applications. Its primary function is to provide a single, unified entry point for all incoming requests, which are then intelligently routed to the appropriate downstream microservice, such as the ListingService, OrderService, or UserService. This centralization simplifies client interactions and decouples clients from the internal topology and scaling instances of the individual services. For SynopsisSI, YARP (Yet Another Reverse Proxy), a Microsoft-built toolkit, was chosen for its seamless integration with the .NET 9 ecosystem, high performance, and flexible configuration capabilities primarily managed via appsettings.json.

The Role of an API Gateway

For the SynopsisSI platform, the API Gateway functions as a reverse proxy, strategically positioned as an intermediary between client applications and the suite of backend microservices. Its fundamental role is to abstract the inherent complexity of the underlying microservice ecosystem from the client, presenting a cohesive and simplified API surface.
A primary responsibility of the Gateway is to act as this single point of entry. All client requests are directed to the Gateway, which then assumes the task of intelligent request routing. Based on characteristics of the incoming request, such as the path, headers, or other attributes, the Gateway forwards the request to the appropriate downstream microservice. This routing mechanism effectively decouples clients from the internal network locations and the dynamic scaling instances of the individual microservices.
Beyond basic routing, an API Gateway can orchestrate more complex interactions, such as request aggregation, where a single client request might trigger multiple calls to backend services, with the Gateway consolidating the responses. The Gateway also serves as an ideal centralized location for managing various cross-cutting concerns common to multiple services.

These include SSL termination, initial authentication and coarse-grained authorization, rate limiting and throttling, caching of common responses, and providing a unified point for logging all API traffic and collecting top-level metrics.

Choice of Reverse Proxy

YARP (Yet Another Reverse Proxy) was selected for implementing the API Gateway in SynopsisSI. As a toolkit built by Microsoft on ASP.NET Core, YARP offers several advantages within the .NET-centric ecosystem of this project.

While YARP is a powerful toolkit, it's important to note it's not a full-featured, out-of-the-box API Management solution like some commercial or comprehensive cloud offerings. Implementing very advanced features like a developer portal or integrated billing would require additional effort. However, for SynopsisSI's core needs of reverse proxying, flexible routing, and a platform for essential cross-cutting concerns, YARP provides a highly capable foundation.

Implementation of SynopsisSI.Gateway

The SynopsisSI.Gateway is an ASP.NET Core project utilizing the Yarp.ReverseProxy NuGet package. Its core setup in Program.cs is streamlined to register YARP's services and load its operational parameters (routes and clusters) from appsettings.json.

Excerpt from SynopsisSI.Gateway/Program.cs:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));
var app = builder.Build();
// ... (development middleware) ...
app.MapReverseProxy();
app.Run();
```

The routing logic is defined in the ReverseProxy section of SynopsisSI.Gateway/appsettings.json, structured into Routes and Clusters. Routes match incoming request paths (e.g., /api/listings/{**catch-all}) and map them via a ClusterId to a corresponding Cluster. Clusters define the destination microservices, using Docker service names (e.g., http://listingservice.api:80) for internal routing within the Docker network and can include configurations for active health checking of these downstream services.

This declarative configuration allows for clear management of how client requests are directed to the appropriate backend microservices like ListingService, OrderService, and UserService. The {**catch-all} segment in route paths ensures that all sub-paths and query strings are correctly forwarded.

Handling Cross-Cutting Concerns

As SynopsisSI matures, the API Gateway is the ideal location for centralizing several cross-cutting concerns. This would involve adding custom ASP.NET Core middleware to the gateway's request pipeline. Conceptually, SynopsisSI.Gateway could manage SSL termination (centralizing certificate management), perform initial lightweight request validation, and implement rate limiting policies to protect downstream services, thereby keeping the individual microservices focused on their core business logic.

A primary security role for the API Gateway is to serve as the first line of defense for authentication. Once a dedicated AuthService (or the UserService) issues JSON Web Tokens (JWTs), client applications will send these tokens with their requests. The SynopsisSI.Gateway can be configured with JWT Bearer authentication middleware to validate these tokens (signature, issuer, audience, expiry). If a token is invalid, the gateway can reject the request immediately with a 401 Unauthorized response. This prevents unauthenticated traffic from reaching backend services and allows for centralized enforcement of initial authentication checks.

Event-Driven Communication

To foster loose coupling and enhance resilience between the microservices in SynopsisSI, particularly for operations that span service boundaries or do not require immediate synchronous responses, an event-driven architectural approach is adopted. This relies on asynchronous messaging, for which RabbitMQ has been chosen as the message broker, and MassTransit is utilized as a .NET abstraction layer to simplify the complexities of message publishing and consumption.

In a distributed system like SynopsisSI, direct synchronous calls between all services for every interaction can lead to tight coupling and reduced fault tolerance; if one service is slow or unavailable, it can cause cascading failures in services that depend on it. Asynchronous messaging helps mitigate these issues. For instance, when an order is placed in the OrderService, instead of directly calling the ListingService to update stock (which might fail if ListingService is temporarily busy or down), OrderService can publish an event. The ListingService can then subscribe to this event and process it when it's able, ensuring that the order placement itself is not blocked by a transient issue in a downstream, non-critical (for the immediate order creation) service. This promotes loose coupling, as services don't need direct knowledge of each other's immediate availability or API contracts for event-based interactions, and resilience, as messages can be queued and processed later if a consuming service is temporarily unavailable.

Choice of RabbitMQ as a Message Broker

RabbitMQ is a mature, widely adopted, and robust open-source message broker that implements the Advanced Message Queuing Protocol (AMQP). It provides flexible routing capabilities (through exchanges and queues), message persistence, acknowledgments, and support for various messaging patterns, making it a suitable choice for handling inter-service events in SynopsisSI.

MassTransit is a free, open-source distributed application framework for .NET that simplifies the creation of applications and services that use message-based communication. It provides a high-level abstraction over various message transports like RabbitMQ, Azure Service Bus, etc. For SynopsisSI, using MassTransit offers:

- Simplified publisher/consumer setup.
- Automatic exchange and queue creation based on conventions or explicit configuration.
- Built-in support for retry mechanisms, error handling (e.g., moving messages to error queues), and sagas (for more complex distributed transactions).
- Easy integration with dependency injection.

Case Study: The OrderPlacedEvent Flow

The process of an order being placed and the subsequent notification to the ListingService to update the item's status serves as a prime example of event-driven communication in SynopsisSI.

An OrderPlacedEvent Data Transfer Object (DTO) is defined in a shared library (SynopsisSI.Shared.Events) to ensure a consistent message contract between the publishing service (OrderService) and any consuming services (ListingService).

```
public class OrderPlacedEvent
{
    public string EventId { get; set; } = Guid.NewGuid().ToString();
    public DateTime Timestamp { get; set; } = DateTime.UtcNow;
    public string OrderId { get; set; } = string.Empty;
    public string BuyerId { get; set; } = string.Empty;
    public List<OrderItemDetail> Items { get; set; } = new();
    // ... other properties like TotalAmount, Currency ...
}
public class OrderItemDetail { /* ListingId, Quantity, etc. */ }
```

After the OrderService successfully creates and commits an order to its database, the PlaceOrderCommandHandler uses an injected IMessageBusPublisher (which is implemented by MassTransitMessageBusPublisher in OrderService.Infrastructure) to publish the OrderPlacedEvent.

```
var orderPlacedEvent = new OrderPlacedEvent { /* ... populate with order details ..
await _messageBusPublisher.PublishAsync(orderPlacedEvent, cancellationToken);
_logger.LogInformation("OrderPlacedEvent published for OrderId: {OrderId}", order.I
```

The Program.cs of OrderService.API configures MassTransit to connect to RabbitMQ and makes IPublishEndpoint (used by IMessageBusPublisher) available via DI.

The ListingService subscribes to the OrderPlacedEvent. An OrderPlacedEventConsumer class is implemented in ListingService.Application, which implements MassTransit's IConsumer<OrderPlacedEvent> interface.

```
public class OrderPlacedEventConsumer : IConsumer<OrderPlacedEvent>
{
    private readonly IUnitOfWork _listingUnitOfWork;
    // ... constructor and logger ...
    public async Task Consume(ConsumeContext<OrderPlacedEvent> context)
    {
        var orderEvent = context.Message;
        _logger.LogInformation("ListingService: OrderPlacedEvent received for Orde
        foreach (var item in orderEvent.Items)
        {
            var listing = await _listingUnitOfWork.Listings.GetByIdAsync(item.List
            if (listing != null && listing.Status == ListingStatus.Available)
            {
                listing.MarkAsSold(orderEvent.OrderId); // Domain logic to update
                await _listingUnitOfWork.Listings.UpdateAsync(listing, context.Can
            }
        }
        await _listingUnitOfWork.SaveChangesAsync(context.CancellationToken); //
    }
```

The Program.cs of ListingService.API configures MassTransit to connect to RabbitMQ and sets up a "receive endpoint" (which corresponds to a queue in RabbitMQ) that binds to the OrderPlacedEvent and dispatches it to the OrderPlacedEventConsumer.

This event-driven flow means that the update to a listing's status (e.g., marking it as "Sold") occurs after the order is initially confirmed. There's a brief period where the order might be placed, but the listing status hasn't yet been updated. This is known as eventual consistency. The system will become consistent, but not instantaneously across all service boundaries. This is a fundamental trade-off in distributed systems for achieving higher availability and decoupling. Business processes and user interface designs may need to account for this (e.g., a listing might temporarily appear available even if an order is processing).

Considerations and Challenges

While offering significant benefits, an event-driven architecture introduces its own set of considerations for SynopsisSI. These include ensuring message delivery guarantees (e.g., at-least-once delivery), handling message ordering if it's critical for certain workflows, managing failed messages (e.g., through dead-letter queues and retry mechanisms provided by MassTransit/RabbitMQ), and the increased complexity of debugging and tracing flows across multiple asynchronous services (which makes comprehensive observability with tools like OpenTelemetry even more vital). Careful design of event contracts and consumer idempotency is also crucial to prevent unintended side effects from reprocessed messages.

Benefits and Considerations of Chosen Integration Strategies

The integration strategies adopted for SynopsisSI—primarily the API Gateway pattern implemented with YARP and an event-driven, asynchronous messaging approach using RabbitMQ and MassTransit—are central to achieving the goals of a decoupled and scalable microservice architecture.

The API Gateway provides significant benefits by acting as a single, managed entry point for all client interactions. This simplifies the client-facing interface, decouples clients from the internal microservice topology, and offers a strategic location for implementing cross-cutting concerns such as SSL termination, request routing, and potentially initial authentication checks or rate limiting. The choice of YARP, being .NET native, allows for deep integration and customization within the existing technology stack. However, the gateway itself becomes a critical component that must be designed for high availability and scalability to avoid becoming a bottleneck.

Event-driven communication between services like OrderService and ListingService (e.g., via an OrderPlacedEvent) fosters autonomy and resilience. Services can operate and evolve independently, and the failure of one service (like ListingService being temporarily unavailable) does not necessarily halt the operations of another (like OrderService accepting new orders). This asynchronicity, managed by RabbitMQ and simplified by MassTransit, allows services to process information at their own pace and handle transient failures more gracefully. The primary consideration with this pattern is eventual consistency: data across different microservices (like an order's state and a listing's availability) becomes consistent over time, not instantaneously within a single distributed transaction. This requires careful design of business processes and user interfaces to manage potentially temporary inconsistencies. Furthermore, an event-driven system introduces the operational overhead of managing a message broker

and requires robust strategies for message retries, dead-lettering, and ensuring consumer idempotency.

For SynopsisSI, these strategies were chosen to prioritize service autonomy, scalability, and resilience, accepting the trade-off of eventual consistency for key cross-service workflows. The benefits of allowing services to evolve independently and the system's ability to degrade gracefully are seen as outweighing the complexities introduced by asynchronous messaging and the need to manage an API Gateway.

Conclusion

This paper has detailed the architectural approach to integrating the SynopsisSI e-commerce platform, transitioning from a conceptual monolithic design to a microservice-based system. The core focus has been on the practical application of an API Gateway (YARP) and event-driven messaging (RabbitMQ/MassTransit) as primary system integration strategies.

The implementation of the SynopsisSI.Gateway successfully demonstrates its role in providing a unified ingress point and efficiently routing client requests to the appropriate backend microservices (ListingService, OrderService, UserService). This centralizes access control and simplifies the client's view of the distributed system.

The adoption of event-driven communication, exemplified by the OrderPlacedEvent flow between OrderService and ListingService, highlights the significant benefits of service decoupling and enhanced resilience. This asynchronous pattern allows services to operate with greater autonomy and better handle transient failures. The choice of RabbitMQ as the message broker and MassTransit as the .NET abstraction layer proved effective in implementing this pattern.

The project successfully established a foundational microservice architecture where core e-commerce functionalities are handled by independent services. The integration through the API Gateway and the event bus promotes scalability and maintainability. Key lessons learned include the importance of clear API contracts for synchronous communication via the gateway and well-defined event schemas for asynchronous messaging. The primary trade-off accepted is that of eventual consistency, which is inherent in such decoupled, event-driven systems.

Future work for SynopsisSI in the realm of system integration would naturally involve expanding the use of these patterns as more microservices (e.g., PaymentService, NotificationService, ReviewService) are introduced. This will necessitate the design of more complex Saga patterns to manage distributed transactions across multiple services, ensuring data integrity for critical business operations that cannot be contained within a single service's local transaction. Further enhancements would include implementing comprehensive distributed tracing and observability (e.g., via OpenTelemetry) to monitor requests and event flows across the entire system, adding advanced security policies and rate limiting at the API Gateway, and systematically applying resilience patterns like circuit breakers and retries for all inter-service communication. These future steps will build upon the robust integration

foundation established in this project, further enhancing the scalability and resilience of the SynopsisSI platform.